Superposition with First-Class Booleans and Inprocessing Clausification

Visa Nummelin[1], Alexander Bentkamp[1], Sophie Tourret[2,3], and Petar Vukmirović[1]

[1] Vrije Universiteit Amsterdam, Amsterdam, The Netherlands
visa.nummelin@vu.nl a.bentkamp@vu.nl p.vukmirovic@vu.nl
[2] Université de Lorraine, CNRS, Inria, LORIA, Nancy, France
sophie.tourret@inria.fr
[3] Max-Planck-Institut für Informatik, Saarland Informatics Campus, Saarbrücken, Germany

Abstract. We present a complete superposition calculus for first-order logic with an interpreted Boolean type. The design of our calculus was driven by three motivating factors. First, the proof of our calculus's completeness lays the foundation for complete calculi in more expressive logics with Booleans, such as higher-order logic. Second, since clausification can make some simple problems hard to prove, our calculus works directly on formulas. Last, we avoid the costly encoding of the theory of Booleans to first-order logic. We evaluate our calculus using the Zipperposition theorem prover, and observe that, with no tuning of heuristic parameters, our approach is on a par with the state-of-the-art approach.

1 Introduction

Superposition is a state-of-the-art calculus for equational first-order logic, which operates on problems given in clausal normal form. Its immense success made clausification preprocessing a predominant mechanism in modern automatic theorem proving. However, clausification is not without drawbacks. In our own experience, we noticed that clausification can transform simple problems, such as $\varphi \to \varphi$ where φ is a large formula, in such a way that its original simplicity is hidden from the superposition calculus. Ganzinger and Stuber's superposition variant [12] operates on clauses that contain formulas as well as terms, and thus replaces clausification preprocessing by inprocessing. Inprocessing clausification allows the powerful simplification engine of superposition to operate on formulas. For example, unit equalities can rewrite formulas before clausification, possibly avoiding many future rewrites after clausification. Whole formulas rather than simple literals can be removed by rules such as subsumption resolution [4].

Another issue with Boolean reasoning in the standard superposition calculus is that, in first-order logic, formulas cannot appear inside terms although this is often desirable for problems coming from software verifiers or proof assistants. Instead, authors of such tools need to resort to translations. Kotelnikov et al. studied effects of these translations in detail. They showed that simple axioms

such as the domain cardinality axiom for Booleans ($\forall(x : o).\, x \approx \top \vee x \approx \bot$) can severely slow down superposition provers. To support more efficient reasoning on problems with first-class Booleans, they describe the FOOL logic, which admits functions that accept arguments of Boolean type and quantification over Booleans. They further describe two approaches to reason in FOOL: The first one [16] requires an additional rule in superposition calculus, whereas the second one [15] is completely based on preprocessing.

Our calculus combines complementary advantages of Ganzinger and Stuber's work and of Kotelnikov et al.'s. Following Kotelnikov et al. (but deviating from Ganzinger and Stuber), our logic (Sect. 2) is similar to FOOL and supports nesting formulas inside terms, as well as quantifying over Booleans. Following Ganzinger and Stuber (but deviating from Kotelnikov et al.), our calculus (Sect. 3) reasons with formulas and supports inprocessing clausification.

In addition to combining the two approaches, our calculus extends them as well. To reduce the number of possible inferences, we generalize Ganzinger and Stuber's Boolean selection functions, which allow us to restrict the Boolean subterms in a clause on which inferences can be performed. The term order requirements of our calculus are less restrictive than Ganzinger and Stuber's. In addition to the lexicographic path order, we also support the Knuth-Bendix order, which is known to work better with superposition in practice.

Our proof of refutational completeness (Sect. 4) lays the foundation for complete calculi in more complex logics with Booleans. Indeed, Bentkamp et al. [8] devised a refutationally complete calculus for higher-order logic based on our completeness theorem. Our theorem incorporates a powerful redundancy criterion that allows for a variety of inprocessing clausification methods (Sect. 5).

We implemented our approach using the Zipperposition theorem prover (Sect. 6). Our evaluation is performed on thousands of problems that target our logic ranging from Sledgehammer-generated, to SMT-LIB, to TPTP benchmarks (Sect. 7). The results are promising: without fine-tuning, our new calculus performs as well as known techniques. Exploring the heuristic choices that our calculus opens should lead to further performance improvements. In addition, we corroborate the claims of Ganzinger and Stuber concerning applicability of formula-based superposition reasoning: We find a set of 17 TPTP problems (out of 1000 randomly selected) that Zipperposition can solve only using the techniques described in this paper. We refer to our technical report [23] for more details on our calculus and the full completeness proof.

2 Logic

Our logic is a first-order logic with an interpreted Boolean type. It is essentially identical to the UF logic of SMT-LIB [5], including the Core theory, but without if-then-else- and let-expressions, which can be supported through simple translations. It also closely resembles Kotelnikov et al.'s FOOL [16], which additionally supports if-then-else- and let-expressions.

Our logic requires an interpreted Boolean type o and allows for an arbitrary number of uninterpreted types. The set of symbols must contain the logical symbols $\mathsf{T}, \bot : o$; $\neg : o \to o$; $\wedge, \vee, \rightarrow : (o \times o) \to o$; and the overloaded symbols $\boldsymbol{\approx}, \boldsymbol{\not\approx} : (\tau \times \tau) \to o$ for each type τ. The logical symbols are printed in bold to distinguish them from the notation used for clauses below. Throughout the paper, we write tuples $(a_1, \ldots, a_n)$ as $\bar{a}_n$ or $\bar{a}$.

The set of *terms* is defined inductively as follows. Every variable is a term. If $\mathsf{f} : \bar{\tau}_n \to \upsilon$ is a symbol and $\bar{t}_n : \bar{\tau}_n$ is a tuple of terms, then the application $\mathsf{f}(\bar{t}_n)$ (or simply f if $n = 0$) is a term of type υ. If x is a variable and $t : o$ a Boolean term, then the quantified terms $\forall x.\, t$ and $\exists x.\, t$ are terms of Boolean type. We view quantified terms modulo α-renaming. A *formula* is a term of Boolean type.

The *head* of a term is f if the term is an application $\mathsf{f}(\bar{t}_n)$; it is x if the term is a variable x; and it is $\forall$ or $\exists$ if the term is a quantified term $\forall x.\, t$ or $\exists x.\, t$. A variable occurrence is *free* in a term if it is not bound by $\forall$ or $\exists$. A term is *ground* if it contains no free variables.

A literal is an equation $s \approx t$ or a disequation $s \not\approx t$. We write $s \mathrel{\dot{\approx}} t$ for a literal that can be either an equation or a disequation. Unlike terms constructed using the function symbols $\boldsymbol{\approx}$ and $\boldsymbol{\not\approx}$, literals are unordered—i.e., $s \mathrel{\dot{\approx}} t$ and $t \mathrel{\dot{\approx}} s$ denote the same literal. A clause $L_1 \vee \cdots \vee L_n$ is a finite multiset of literals L_j. The empty clause is written as $\bot$. Terms t of Boolean type are not literals. They must be encoded as $t \approx \mathsf{T}$ and $t \approx \bot$, which we call *predicate literals*. Both are considered positive literals because they are equations, not disequations.

We have considered to exclude negative literals $s \not\approx t$ by encoding them as $(s \boldsymbol{\approx} t) \approx \bot$, following Ganzinger and Stuber. However, this approach requires an additional term order condition to make the conclusion of equality factoring small enough, excluding the Knuth–Bendix order. To support both the Knuth–Bendix order and the lexicographic path order, we allow negative literals. Regardless, our simplification mechanism will allow us to simplify negative literals of the form $t \not\approx \bot$ and $t \not\approx \mathsf{T}$ into $t \approx \mathsf{T}$ and $t \approx \bot$, respectively, thereby eliminating redundant representations of predicate literals.

The semantics is a straightforward extension of standard first-order logic, as in Kotelnikov et al.'s FOOL logic. Some of our calculus rules introduce Skolem symbols, which are intended to be interpreted as witnesses for existentially quantified terms. Still, our semantics treats them as uninterpreted symbols. To achieve a satisfiability-preserving calculus, we assume that these symbols do not occur in the input problem. More precisely, we inductively extend the signature of the input problem by a symbol $\mathsf{sk}_{\exists z.\, t} : \bar{\tau} \to \upsilon$ for each term of the form $\exists z.\, t$ over the extended signature, where υ is the type of z and $\bar{\tau}$ are the types of the free variables occurring in $\exists z.\, t$, in order of first appearance.

3 The Calculus

Following standard superposition, our calculus employs a term order and a literal selection function to restrict the search space. To accommodate for quantified Boolean terms, we impose additional requirements on the term order. To support

flexible reasoning with Boolean subterms, in addition to the literal selection function, we introduce a Boolean subterm selection function.

Term Order The calculus is parameterized by a strict well-founded order $\succ$ on ground terms that fulfills: (O1) $u \succ \bot \succ \top$ for any term u that is not $\top$ or $\bot$; (O2) $\forall x.\, t \succ \{x \mapsto u\}t$ and $\exists x.\, t \succ \{x \mapsto u\}t$ for any term u whose only Boolean subterms are $\top$ and $\bot$; (O3) subterm property; (O4) compatibility with contexts (not necessarily below $\forall$ and $\exists$); (O5) totality. The order is extended to literals, clauses, and nonground terms as usual [2]. The nonground order then also enjoys (O6) stability under grounding substitutions.

Ganzinger and Stuber's term order restrictions are similar but incompatible with the Knuth-Bendix order (KBO) [14]. Using an encoding of our terms into untyped first-order logic we describe how both the lexicographic path order (LPO) and the transfinite variant of KBO [18] can satisfy conditions (O1)-(O6).

Our encoding represents bound variables by De Bruijn indices, which become new constant symbols db_n for $n \in \mathbb{N}$. Quantifiers are represented by two new unary function symbols, also denoted by $\forall$ and $\exists$. All other symbols are simply identified with their untyped counterpart. Regardless of symbol precedence or symbol weight assignment, KBO and LPO satisfy properties (O3)–(O6) when applied to the encoded terms because they are ground-total simplification orders. They are even compatible with contexts below quantifiers.

To satisfy (O1) and (O2), let the precedence for LPO be $\top < \bot < \mathsf{f} < \forall < \exists < \mathsf{db}_0 < \mathsf{db}_1 < \cdots$ where f is any other symbol. For KBO, we can use the same symbol precedence and a symbol weight function $\mathcal{W}$ that assigns each symbol ordinal weights (of the form $\omega a + b$ with $a, b \in \mathbb{N}$), where $\mathcal{W}(\top) = \mathcal{W}(\bot) = 1$, $\mathcal{W}(\forall) = \mathcal{W}(\exists) = \omega$, and $\mathcal{W}(\mathsf{f}) \in \mathbb{Z}_+$ for any other symbol f.

Selection and Eligibility Following an idea of Ganzinger and Stuber, we parameterize our calculus with two selection functions: one selecting literals and one selecting Boolean subterms. Our selection functions have weaker restrictions:

Definition 1 (Selection functions). The calculus is parameterized by a literal selection function *FLSel* and a Boolean subterm selection function *FBSel*. The function *FLSel* maps each clause to a subset of its literals. The selection function *FBSel* maps each clause to a subset of its Boolean subterms. The literals $FLSel(C)$ and the subterms $FBSel(C)$ are called *selected* in C. The following selection restrictions apply: (S1) A literal can only be selected if it is negative or of the form $s \approx \bot$; (S2) A Boolean subterm can only be selected if it is not $\top$, $\bot$, or a variable; (S3) A Boolean subterm can only be selected if its occurrence is not below a quantifier; and (S4) The topmost terms on either side of a positive literal cannot be selected.

The interplay of maximality with respect to term order, literal and Boolean selection functions gives rise to a new notion of eligibility:

Definition 2 (Eligibility). A literal L is (*strictly*) *eligible* w.r.t. a substitution σ in C if it is selected in C or there are no selected literals and no selected Boolean

subterms in C and σL is (strictly) maximal in σC. Thus, a selected literal is strictly eligible. The eligible subterms of a clause C w.r.t. a substitution σ are inductively defined as follows: (E1) Any selected subterm is eligible. (E2) If a literal $s \mathrel{\dot{\approx}} t$ with $\sigma s \not\preceq \sigma t$ is either eligible and negative or strictly eligible and positive, then s is eligible. (E3) If a subterm is eligible and its head is not $\boldsymbol{\approx}$, $\not\boldsymbol{\approx}$, $\forall$, or $\exists$, all of its direct subterms are also eligible. (E4) If a subterm is eligible and of the form $s \boldsymbol{\approx} t$ or $s \not\boldsymbol{\approx} t$, then s is eligible if $\sigma s \not\preceq \sigma t$ and t is eligible if $\sigma s \not\succeq \sigma t$. The substitution σ is left implicit if it is the identity substitution.

The Core Inference Rules The following inference rules form our calculus:

$$\frac{\overbrace{D' \vee t \approx t'}^{D} \quad C[u]}{\sigma(D' \vee C[t'])}\ \textsc{Sup} \qquad \frac{\overbrace{C' \vee s \approx t}^{C}}{\sigma C'}\ \bot\textsc{Elim}$$

$$\frac{\overbrace{C' \vee u \not\approx u'}^{C}}{\sigma C'}\ \textsc{Irrefl} \qquad \frac{C[u]}{C[\bot] \vee u \approx \top}\ \textsc{BoolHoist}$$

$$\frac{\overbrace{C' \vee u' \approx v' \vee u \approx v}^{C}}{\sigma(C' \vee v \not\approx v' \vee u \approx v')}\ \textsc{EqFact} \qquad \frac{C[s \boldsymbol{\approx} t]}{C[\bot] \vee s \approx t}\ \boldsymbol{\approx}\textsc{Hoist}$$

$$\frac{C[u]}{\sigma C[t']}\ \textsc{BoolRw} \qquad \frac{C[s \not\boldsymbol{\approx} t]}{C[\top] \vee s \approx t}\ \not\boldsymbol{\approx}\textsc{Hoist}$$

$$\frac{C[\exists z.\, v]}{C[\{z \mapsto \mathsf{sk}_{\exists z.v}(\bar{y})\}v]}\ \exists\textsc{Rw} \qquad \frac{C[\forall x.\, t]}{C[\bot] \vee \{x \mapsto y\}t \approx \top}\ \forall\textsc{Hoist}$$

$$\frac{C[\forall z.\, v]}{C[\{z \mapsto \mathsf{sk}_{\exists z.\neg v}(\bar{y})\}v]}\ \forall\textsc{Rw} \qquad \frac{C[\exists x.\, t]}{C[\top] \vee \{x \mapsto y\}t \approx \bot}\ \exists\textsc{Hoist}$$

The rules are subject to the following side conditions:

Sup 1. $\sigma = \mathrm{mgu}(t, u)$; 2. u is not a variable; 3. $\sigma t \not\preceq \sigma t'$; 4. $D \prec C[u]$; 5. u is eligible in C w.r.t. σ; 6. $t \approx t'$ is strictly eligible in D w.r.t. σ; 7. the head of t is not a logical symbol; 8. if $\sigma t' = \bot$, the subterm u is at the top level of a positive literal.

Irrefl 1. $\sigma = \mathrm{mgu}(u, u')$; 2. $u \not\approx u'$ is eligible in C w.r.t. σ.

EqFact 1. $\sigma = \mathrm{mgu}(u, u')$; 2. $\sigma u \not\approx t \notin \sigma C$ for any term t; 3. no Boolean subterm and no literal is selected in C; 4. σu is a maximal term in σC; 5. σv is maximal in $\{t \mid \sigma u \approx t \in \sigma C\}$.

BoolRw 1. $\sigma = \mathrm{mgu}(t, u)$; 2. u is not a variable; 3. u is eligible in C w.r.t. σ; 4. (t, t') is one of the following pairs, where x is a fresh variable: $(\neg\bot, \top)$,

$(\neg\top, \bot)$, $(\bot \land \bot, \bot)$, $(\top \land \bot, \bot)$, $(\bot \land \top, \bot)$, $(\top \land \top, \top)$, $(\bot \lor \bot, \bot)$, $(\top \lor \bot, \top)$, $(\bot \lor \top, \top)$, $(\top \lor \top, \top)$, $(\bot \to \bot, \top)$, $(\top \to \bot, \bot)$, $(\bot \to \top, \top)$, $(\top \to \top, \top)$, $(x \approx x, \top)$, $(x \not\approx x, \bot)$.

$\star$Rw (where $\star \in \{\forall, \exists\}$) 1. v is a term that may contain the loose bound variable z; 2. $\bar{y}$ are the free variables occurring in $\exists z.\, v$ and $\forall z.\, v$, respectively, in order of first appearance; 3. the indicated subterm is eligible in C; 4. for $\forall$Rw, $C[\top]$ is not a tautology; 5. for $\exists$Rw, $C[\bot]$ is not a tautology.

$\bot$Elim 1. $\sigma = \mathrm{mgu}(s \approx t, \bot \approx \top)$; 2. $s \approx t$ is strictly eligible in C w.r.t. σ.

BoolHoist 1. u is a Boolean term whose head is an uninterpreted predicate; 2. u is eligible in C; 3. u is not a variable; 4. u is not at the top level of a positive literal.

$\star$Hoist (where $\star \in \{\approx, \not\approx, \forall, \exists\}$) 1. the indicated subterm is eligible in C; 2. y is a fresh variable.

Rationale for the Rules Our calculus is a graceful generalization of superposition: if the input clauses do not contain any Boolean terms, it coincides with standard superposition. In addition to the standard superposition rules Sup, Irrefl, and EqFact, our calculus contains various rules to cope with Booleans. For each logical symbol and quantifier, we must consider the case where it is true and the case where it is false. Whenever possible, we prefer rules that rewrite the Boolean subterm in place (with names ending in Rw). When this cannot be done in a satisfiability-preserving way, we resort to rules hoisting the Boolean subterm into a dedicated literal (with names ending in Hoist). For terms headed by an uninterpreted predicate, the rule BoolHoist only copes with the case that the term is false. If it is true, we rely on Sup to rewrite it to $\top$ eventually.

Example 3. The clause $\mathsf{a} \land \neg\mathsf{a} \approx \top$ can be refuted by the core inferences as follows. First we derive $\mathsf{a} \approx \top$ (displayed on the left) and then we use it to derive $\bot$ (displayed on the right). In this and the following example, we assume eager selection of literals whenever the selection restrictions allow it.

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\mathsf{a} \land \neg\mathsf{a} \approx \top}{\bot \land \neg\mathsf{a} \approx \top \lor \mathsf{a} \approx \top}\,\textsc{BoolHoist}}{\bot \land \neg\bot \approx \top \lor \mathsf{a} \approx \top \lor \mathsf{a} \approx \top}\,\textsc{BoolHoist}}{\bot \land \top \approx \top \lor \mathsf{a} \approx \top \lor \mathsf{a} \approx \top}\,\textsc{BoolRw}}{\bot \approx \top \lor \mathsf{a} \approx \top \lor \mathsf{a} \approx \top}\,\textsc{BoolRw}}{\mathsf{a} \approx \top \lor \mathsf{a} \approx \top}\,\bot\textsc{Elim}}{\top \not\approx \top \lor \mathsf{a} \approx \top}\,\textsc{EqFact}}{\mathsf{a} \approx \top}\,\textsc{Irrefl}$$

$$\dfrac{\dfrac{\dfrac{\mathsf{a} \approx \top \quad \dfrac{\mathsf{a} \land \neg\mathsf{a} \approx \top \quad \mathsf{a} \approx \top}{\top \land \neg\mathsf{a} \approx \top}\,\textsc{Sup}}{\top \land \neg\top \approx \top}\,\textsc{Sup}}{\top \land \bot \approx \top}\,\textsc{BoolRw}}{\bot \approx \top}\,\textsc{BoolRw}}{\bot}\,\bot\textsc{Elim}$$

The derivation illustrates how BoolHoist and Sup replace uninterpreted predicates by $\top$ and $\bot$ to allow BoolRw to eliminate the surrounding logical symbols.

Example 4. The clause $(\exists x.\, \forall y.\, y \not\approx x) \approx \mathsf{T}$ can be refuted as follows:

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{(\exists x.\, \forall y.\, y \not\approx x) \approx \mathsf{T}}{(\forall y.\, y \not\approx \mathsf{sk}_{\exists x.\forall y.y\not\approx x}) \approx \mathsf{T}}\,\exists\textsc{Rw}}{\bot \approx \mathsf{T} \vee (y' \not\approx \mathsf{sk}_{\exists x.\forall y.y\not\approx x}) \approx \mathsf{T}}\,\forall\textsc{Hoist}}{(y' \not\approx \mathsf{sk}_{\exists x.\forall y.y\not\approx x}) \approx \mathsf{T}}\,\bot\textsc{Elim}}{\bot \approx \mathsf{T}}\,\not\approx\textsc{Rw}}{\bot}\,\bot\textsc{Elim}$$

Redundancy Criterion In standard superposition, a clause is defined as redundant if all of its ground instances follow from smaller ground instances of other clauses. We keep this definition, but use a nonstandard notion of ground instances. In our completeness proof, this new notion of ground instances ensures that ground instances of the conclusion of ∀Hoist, ∃Hoist, ∀Rw, and ∃Rw inferences are smaller than the corresponding instances of their premise by property (O2).

Definition 5 (Redundancy of clauses). The *ground instances* of a clause C are all clauses of the form γC where γ is a substitution such that for all variables x, the only Boolean subterms of γx are $\bot$ and T. A ground clause C is *redundant* with respect to a ground clause set N if there exist clauses $C_1, \ldots, C_k \in N$ such that we have $C_1, \ldots, C_k \models C$ and $C \succ C_i$ for all $1 \leq i \leq k$. A nonground clause C is redundant w.r.t. clauses N if C is strictly subsumed by a clause in N or every ground instance of C is redundant w.r.t. ground instances of N.

In standard superposition, an inference is defined as redundant if all of its ground instances are, and a ground inference is defined as redundant if its conclusion follows from other clauses smaller than the main premise. We keep this definition as well, but we use a nonstandard notion of ground instances for some of the Boolean rules. In our report, we define a slightly stronger variant of inference redundancy via an explicit ground calculus, but the following notion is also strong enough to justify the few prover optimizations based on inference redundancy we know from the literature (e.g., simultaneous superposition [7]).

Definition 6 (Redundancy of inferences). A *ground instance* of a ∀Hoist, ∃Hoist, ∀Rw, or ∃Rw inference is an inference obtained by applying a grounding substitution to premise and conclusion, regardless of whether the result is a valid ∀Hoist, ∃Hoist, ∀Rw, or ∃Rw inference. A *ground instance* of an inference ι of other rules is an inference ι' of the same rule such that premises and conclusion of ι' are ground instances of the respective premises and conclusion of ι. For ι', we use selection functions that select the ground literals and Boolean subterms corresponding to the ones selected in the nonground premises. A ground inference with main premise C, side premises $C_1, \ldots, C_n$, and conclusion D is *redundant* with respect to N if there exist clauses $D_1, \ldots, D_k \prec C$ in N such that $D_1, \ldots, D_k, C_1, \ldots, C_n \models D$. A nonground inference is redundant if all its ground instances are redundant.

A clause set N is *saturated* if every inference from N is redundant w.r.t. N.

Simplification Rules The redundancy criterion is a graceful generalization of the criterion of standard superposition. Thus, the standard simplification and deletion rules, such as deletion of trivial literals and clauses, subsumption, and demodulation, can be justified. Demodulation below quantifiers is justified if the term order is compatible with contexts below quantifiers.

Some calculus rules can act as simplifications. $\bot$Elim can always be a simplification. Given a clause on which both $\star$Hoist and $\star$Rw apply, where $\star \in \{\forall, \exists\}$, the clause can be replaced by the conclusions of these rules. If $\star$Rw does not apply because of condition 4 or 5, $\star$Hoist alone can be a simplification.

While experimenting with our implementation, we have observed that the following simplification rule can substantially shorten proofs:

$$\frac{s \not\approx t \vee C[s]}{s \not\approx t \vee C[t]}\,\textsc{LocalRw}$$

In this rule, we require $s \succ t$. Even though we discovered the rule independently, a similar rule was already available in the Vampire prover [17].

Interpreting literals of the form $s \approx \top$ and $s \approx \bot$ as $s \not\approx \bot$ and $s \not\approx \top$, respectively, we can apply the rule even to these positive literals. This especially comes in handy with rules such as BoolHoist. Consider the clause $C = \mathsf{p}^i(\bot) \approx \bot \vee \mathsf{q} \approx \bot$, assume no literal is selected and the Boolean selection function always selects a subterm $\mathsf{p}(\bot)$. Applying BoolHoist to C we get $C_1 = \mathsf{p}(\bot) \approx \top \vee \mathsf{p}^{i-1}(\bot) \approx \bot \vee \mathsf{q} \approx \bot$. Clause C_1 can then be simplified to a tautological clause $\mathsf{p}(\bot) \approx \top \vee \mathsf{p}(\bot) \approx \bot \vee \mathsf{q} \approx \bot$ using $i-2$ LocalRw steps. If we did not use LocalRw, BoolHoist would produce $i-2$ intermediary clauses starting from C, none of which would be recognized as a tautology.

Many rules of our calculus replace subterms with $\top$ or $\bot$. After this replacement, resulting terms can be simplified using Boolean equivalences that specify the behavior of logical operations on $\top$ and $\bot$. To this end, we use the rule BoolSimp [31], similar to simp of Leo-III [25, Sect. 4.2.1.]:

$$\frac{C[s]}{C[t]}\,\textsc{BoolSimp}$$

This rule replaces s with t whenever $s \approx t$ is contained in a predefined set of tautological equations. In addition to all equations that Leo-III uses for simp, we also include more complex ones, such as $(\neg u \rightarrow u) \approx u$ and $(u_1 \rightarrow \cdots \rightarrow u_n \rightarrow v_1 \vee \cdots \vee v_m) \approx \top$ where $u_i = v_j$ for some i and j. The exhaustive list is given in our technical report. Using BoolSimp and $\bot$Elim, Example 3 can be solved in two simplification steps instead of twelve inference steps.

4 Refutational Completeness

Our calculus is not sound because of the introduction of Skolem symbols in $\forall$Rw and $\exists$Rw but it preserves satisfiability. It is also dynamically refutationally complete:

Completeness Theorem 7. *Let S_0 be an unsatisfiable set of clauses. Let $(S_i)_i$ be a fair derivation, i.e., a sequence $S_0, S_1, \ldots$ where every inference from the clauses in $\bigcup_{i=1}^{\infty} \bigcap_{j>i} S_j$ is computed and added to some S_j, or at least becomes redundant w.r.t. S_j. Then $\bot \in S_i$ for some i.*

We outline some key parts of the proof here but refer to our technical report [23] for the details. In our proof, we first define a ground calculus and prove it complete. Devising suitable ground analogues of the rules ∀Rw and ∃Rw was difficult because the arguments of the Skolems depend on the variables occurring in the premise. Therefore, we parameterize the ground calculus by a function that provides ground Skolem terms in the ground versions of these rules. When lifting the completeness result to the nonground level, we instantiate the parameter with a specific function that allows us to lift the ∀Rw and ∃Rw inferences.

To prove the ground calculus complete, we employ the framework for reduction of counterexamples [3]. It requires us to construct an interpretation $\mathcal{I}$ given a saturated unsatisfiable clause set that does not contain $\bot$. Then we must show that any counterexample—i.e., a clause that does not hold in $\mathcal{I}$—can be reduced to a smaller ($\prec$) counterexample by some inference.

The interpretation $\mathcal{I}$ is defined by a normalizing rewrite system as in the standard completeness proof of superposition. To ensure a correct semantics, we incrementally add Boolean rewrite rules along with the rules produced by clauses as usual. Intuitively, if a counterexample can be rewritten by a Boolean rule, we reduce it by a ⋆Hoist or ⋆Rw inference. If it can be rewritten by a rule produced by a clause, we reduce it by a Sup inference.

We derive the dynamic completeness of our nonground calculus using the the saturation framework [33]. It gives us a nonground clause set N to work with. We then have to choose the parameters of our ground calculus such that all of its inferences from the grounding of N are redundant or liftable. We show that inferences rewriting below variables are redundant. Other inferences are shown to be liftable—i.e., they are a ground instance of some nonground inference from N.

5 Inprocessing Clausification Methods

Our calculus makes preprocessing clausification unnecessary: A problem specified by a formula f can be represented as a clause $f \approx \top$. Our redundancy criterion allows us to add various sets of rules to steer the clausification inprocessing.

Without any additional rules, our core calculus rules perform all the necessary reasoning with formulas. We call this method *inner delayed clausification* because the calculus rules tend to operate on the inner Boolean subterms first.

The *outer delayed clausification* method adds the following rules to the calculus, which are guided by the outermost logical symbols. Let s and t be Boolean terms. Below, we denote literals of the form $s \approx \top$ and $s \not\approx \bot$ as s^+, and literals of the form $s \approx \bot$ and $s \not\approx \top$ as s^-.

$$\frac{s^+ \vee C}{oc(s, C)}\,{+}\textsc{OuterClaus} \qquad \frac{s^- \vee C}{oc(\neg s, C)}\,{-}\textsc{OuterClaus}$$

$$\frac{s \approx t \vee C}{s \approx \bot \vee t \approx \top \vee C \quad s \approx \top \vee t \approx \bot \vee C}\,{\approx}\textsc{OuterClaus}$$

$$\frac{s \not\approx t \vee C}{s \approx \bot \vee t \approx \bot \vee C \quad s \approx \top \vee t \approx \top \vee C}\,{\not\approx}\textsc{OuterClaus}$$

The rules $+$OuterClaus and $-$OuterClaus are applicable to any term s whose head is a logical symbol, whereas the rules $\approx$OuterClaus and $\not\approx$OuterClaus are only applicable if neither s nor t is $\top$ or $\bot$. Clearly, our redundancy criterion allows us to replace the premise of all OuterClaus-rules with their conclusions. Nonetheless, the rules $\approx$OuterClaus and $\not\approx$OuterClaus are not used as simplification rules since destructing equivalences disturbs the syntactic structure of the formulas, as noted by Ganzinger and Stuber [12]. The function $oc(s, C)$ analyzes the shape of the formula s and distributes it over the clause C. For example, $oc(s_1 \rightarrow s_2, C) = \{s_1 \approx \bot \vee s_2 \approx \top \vee C\}$, and $oc(\neg(s_1 \vee s_2), C) = \{s_1 \approx \bot \vee C, s_2 \approx \bot \vee C\}$. This function also replaces quantified terms by either a fresh free variable or a Skolem in the body of the quantified term, depending on the polarity. The full definition of $oc(s, C)$ is specified in our technical report.

A third inprocessing clausification method is *immediate clausification*. It first preprocesses the input problem using a standard first-order clausification procedure such as Nonnengart and Weidenbach's [22]. Then, during the proof search, when a clause C appears on which OuterClaus rules could be applied, we apply the standard clausification procedure on the formula $\forall \bar{x}.\, C$ instead (where $\bar{x}$ are the free variables of C), and replace C with the clausification results. With this method, the formulas are clausified in one step, making intermediate clausification results inaccessible to the simplification machinery.

Renaming Common Formulas Following Tseitin [29], clausification procedures usually rename common formulas to prevent a possible combinatorial explosion caused by naive clausification. In our two delayed clausification methods, we realize this idea using the following rule. Given a *definition clause* R, i.e., a clause of the form $\mathsf{p}(\bar{x}) \approx f$:

$$\frac{C_1[\sigma_1 f] \quad \cdots \quad C_n[\sigma_n f]}{C_1[\sigma_1 \mathsf{p}(\bar{x})] \quad \cdots \quad C_n[\sigma_n \mathsf{p}(\bar{x})] \qquad R_1 \quad \cdots \quad R_m}\,\textsc{Rename}$$

Here, the formula f has a logical head, $\bar{x}$ are the free variables in f, p is a fresh symbol, σ_i is a substitution, and the clauses $R_1, \ldots, R_m$ are the result of simplifying R as described below. The rule avoids exponential explosion by replacing n positions in which results of f's clausification will appear into a single

position in R. Optimizations such as polarity-aware renaming [22, Sect. 4] also apply to Rename.

Several issues arise with Rename as an inprocessing rule. We need to ensure that in R, $f \succ \mathsf{p}(\bar{x})$, since otherwise demodulation might reintroduce a formula f in the simplified clauses. This can be achieved by giving the fresh symbol p a precedence smaller than that of all symbols initially present in the problem (other than $\top$ and $\bot$). To ensure the precedence is well founded, the precedence of p must be greater than that of symbols previously introduced by the calculus. For KBO, we additionally set the weight of p to the minimal possible weight.

For Rename to be used as a simplification rule, we need to ensure that the conclusions are smaller than the premises. This is trivially true for all clauses other than the clause R. For example, let $C_i = f \approx \top$ (σ_i is the identity). Clearly, R is larger than C_i. However, we can view the definition clause R as two clauses $R^+ = \mathsf{p}(\bar{x}) \approx \bot \vee f \approx \top$ and $R^- = \mathsf{p}(\bar{x}) \approx \top \vee f \approx \bot$. Then, we can apply a single step of OuterClaus-rules to R^+ and R^- (on their subformula f), which further results in clauses $R_1, \ldots, R_m$. Inspecting OuterClaus-rules, it is clear that $m \leq 4$, which makes enforcing this simplification tolerable. Furthermore, as f is simplified in each of $R_1, \ldots, R_m$, they are smaller than any premise C_i.

Another potential source of a combinatorial explosion in our calculus are formulas that occur deep in the arguments of uninterpreted predicates. Consider the clause $C = \mathsf{p}^i(x) \approx \top \vee \mathsf{q}^j(y) \approx \top$ where $i, j > 2$. If the first and the second literal are eligible in C, any clause $p^{i_1}(x) \approx \top \vee p^{i_2}(\bot) \approx \top \vee \cdots \vee p^{i_k}(\bot) \approx \top \vee \mathsf{q}^{j_1}(y) \approx \top \vee \mathsf{q}^{j_2}(\bot) \approx \top \vee \cdots \vee \mathsf{q}^{j_l}(\bot) \approx \top$ (where $i_1 + \cdots + i_k = i$ and $j_1 + \cdots + j_l = j$) , resulting from multiple BoolHoist applications, can be obtained in many different ways. This explosion can be avoided using the following rule:

$$\frac{s \approx t \vee C}{\mathsf{p}(\overline{x}) \approx \mathsf{T} \vee C \qquad R_1 \quad \cdots \quad R_4} \text{RenameDeep}$$

where p is a fresh symbol, $\bar{x}$ are all free variables occurring in $s \approx t$, the clauses $R_1, \ldots, R_4$ result from simplifying $R = \mathsf{p}(\bar{x}) \approx (s \mathrel{\boldsymbol{\approx}} t)$ as described above, and we impose the same precedence and weight restrictions on p as for Rename. Finally, we require that both $s \approx t$ and C contain deep Booleans where a Boolean subterm $u|_p$ of a term u is a *deep Boolean* if there are at least two distinct proper prefixes q of the position p such that the head of $u|_q$ is an uninterpreted predicate.

Similarly to Rename, the definition clause R can be larger than the premise. As OuterClaus-rules might not apply to $s \mathrel{\boldsymbol{\approx}} t$, we need a different solution:

$$\frac{C[u]}{C[\bot] \vee u \approx \mathsf{T} \quad C[\mathsf{T}] \vee u \approx \bot} \text{BoolHoistSimp}$$

In this rule u is a non-variable Boolean subterm, different from $\top$ and $\bot$, whose indicated occurrence is not in a literal $u \approx b$ where b is $\top$, $\bot$ or a variable. Clearly, both conclusions of BoolHoistSimp are smaller than the premise. As before, observing that R is equivalent to two clauses $R^+ = \mathsf{p}(\bar{x}) \approx \bot \vee s \approx t$ and $R^- = \mathsf{p}(\bar{x}) \approx \top \vee s \not\approx t$, we simplify R^+ and R^- into clauses that are guaranteed

to be smaller than the premise. This is achieved by applying BoolHoistSimp to one of the deep Boolean occurrences in both R^+ and R^-, which produces $R_1, \ldots, R_4$ and reduces the size of resulting clauses enough for them to be smaller than the premise of RenameDeep. The RenameDeep rule can be applied analogously to negative literals $s \not\approx t$.

6 Implementation

Zipperposition [10] is an automatic theorem prover designed for easy prototyping of various extensions of superposition. So far, it has been extended to support induction, arithmetic, and various fragments of higher-order logic. We have implemented our calculus and its extensions described above in Zipperposition.

Zipperposition has long supported λ as the only binder. Because introducing new binders would significantly complicate the implementation, we decided to represent the terms $\forall x.\, t$ and $\exists x.\, t$ as $\forall(\lambda x.\, t)$ and $\exists(\lambda x.\, t)$, respectively.

We introduced a normalized presentation of predicate literals as either $s \approx \top$ or $s \approx \bot$. As Zipperposition previously encoded them as $s \approx \top$ or $s \not\approx \top$, enforcing the new encoding was a source of great implementation effort.

The presentation and implementation of the calculus rules differ in some ways, without affecting completeness. Relying on the presence of BoolSimp, we can simplify the implementation of BoolRw. BoolSimp simplifies logical symbols if one argument is either $\top$ or $\bot$ or if two arguments are identical. Therefore, we need to implement BoolRw only for two cases where BoolSimp does not apply: if all arguments of a logical symbol are distinct variables and if the sides of an equation or disequation are different and unifiable. Justified by our redundancy criterion, the rules BoolHoist and $\star$Hoist simultaneously replace all occurrences of the eligible subterm they act on. For example, applying $\boldsymbol{\approx}$Hoist to $\mathsf{p}(x \boldsymbol{\approx} y) \approx \top \vee \mathsf{q}(x \boldsymbol{\approx} y) \approx \bot$ yields $\mathsf{p}(\bot) \approx \top \vee \mathsf{q}(\bot) \approx \bot \vee x \approx y$. The last two conditions of the $\star$Rw rules are approximated by checking if the affected literal is of the form $\exists z.\, v \approx \bot$ (for $\exists$Rw) or $\forall z.\, v \approx \top$ (for $\forall$Rw). These are variants of the EqFact conditions but was discovered after the evaluation, thus here we perform EqFact inferences even if the maximal literal is selected.

Zipperposition's existing selection functions were not designed with Boolean subterm selection in mind. For instance, a function that selects a literal L with a selectable Boolean subterm s can make s eligible, even if the Boolean selection function did not select s. To mitigate this issue, we can optionally block selection of literals that contain selectable Boolean subterms.

We implemented four Boolean selection functions: selecting the leftmost innermost, leftmost outermost, syntactically largest or syntactically smallest selectable subterm. Ties are broken by selecting the leftmost term. Additionally, we implemented a Boolean selection function that does not select any subterm.

Vukmirović and Nummelin [31, Sect. 3.4] explored inprocessing clausification as part of their pragmatic approach to Boolean reasoning. They describe in detail how the formula renaming mechanism is implemented. We reuse their mechanism, and additionally simplify definition clauses as described in Sect. 5.

7 Evaluation

The goal of our evaluation is to answer the following questions:

1. How does our approach compare to preprocessing?
2. How do the different inprocessing clausification methods compare?
3. Is there an overhead of our calculus on problems without first-class Booleans?
4. What effect do Boolean selection, LocalRw, and BoolHoistSimp have?

We filtered TPTP [27] and SMT-LIB [5] to obtain benchmarks that belong to our logic, but not to first-order logic. In TPTP THF we found 145 such problems and in the UF section of SMT-LIB 5507 such problems. Jasmin Blanchette and Martin Desharnais generated 1253 Sledgehammer problems that target our logic. To measure the overhead of our calculus, we randomly chose 1000 FOF and CNF problems from the TPTP. We restricted our evaluation to this subset because our experiments can already take up to (145+5507+1253+1000) $\times$ #modes $\times$ 300 s $\approx$ 6600 CPU hours. That is over two months on an average quad-core laptop. On StarExec servers, evaluation roughly takes three days under low load. Evaluating on all FOF and CNF problems (around 13 000) could take 2.5 times longer, severely slowing down our work.

SMT-LIB interprets the symbol ite as the standard if-then-else function [5, Sect. 3.7.1]. Whenever a term $s = \mathsf{ite}(t_1, t_2, t_3)$ of type τ occurs in the problem, we replace s with $\mathsf{f}_\tau(t_1, t_2, t_3)$, where f_τ is a fresh symbol denoting the ite function of a particular return type. To comply with SMT-LIB, we add the following axioms: $\forall x\, y.\, \mathsf{f}_\tau(\top, x, y) \approx x$ and $\forall x\, y.\, \mathsf{f}_\tau(\bot, x, y) \approx y$. SMT-LIB allows the use of `let` variable bindings [5, Sect. 3.6.1]. We simply replace each variable with its definition in the body of the `let` bindings.

Currently, out of competing superposition-based provers only Vampire and E support first-order logic with interpreted Booleans, and they do so through preprocessing. Since the main task of this evaluation is to assess the performance of our new calculus, which is implemented only in Zipperposition, we decided not to evaluate the other provers.

We use Zipperposition's first-order portfolio, which invokes the prover sequentially, with up to 13 configurations of heuristic parameters in different time slices. To compare different features, we run different modes that enable a given feature in all of the portfolio configurations. All experiments were performed on the StarExec Iowa servers [26], equipped with Intel Xeon E5-2609 0 CPUs clocked at 2.40 GHz. We set the CPU time limit to 300 s. Figure 1 displays the results. Empty cells indicate that a mode is not evaluated on a benchmark set. An archive with the raw evaluation data is publicly available[4].

A preprocessing transformation that removes all Boolean subterms occurring as arguments of symbols [32, Sect. 8], akin to Kotelnikov et al.'s FOOL clausification approach [15], is implemented in Zipperposition. To answer question 1, we enabled preprocessing and compared it to our new calculus parameterized with the Boolean selection function that selects the smallest selectable subterm.

[4] https://doi.org/10.5281/zenodo.4550787

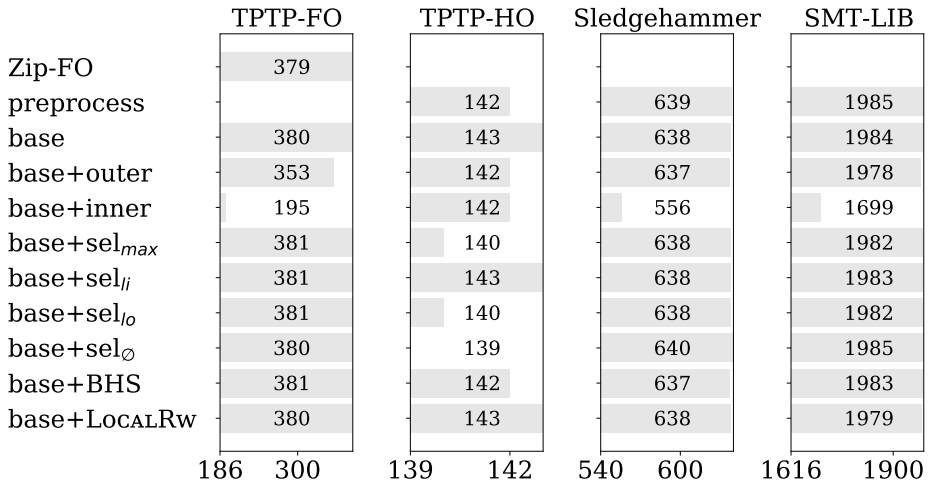

Fig. 1: Number of problems solved per benchmark set and Zipperposition mode. The x-axes start from the number of problems solved by all evaluated modes.

The mode using our new calculus performs immediate inprocessing clausification, and we call it *base*, while the mode that preprocesses Boolean subterms is denoted by *preprocess* in Figure 1.

The obtained results do not give a conclusive answer to question 1. On both TPTP-HO and Sledgehammer problems, some configuration of our new calculus manages to prove one problem more than preprocessing. On SMT-LIB benchmarks, the best configuration of our calculus matches preprocessing. This shows that our calculus performs roughly as well as previously known techniques.

Our *base* mode uses immediate inprocessing clausification. To answer question 2, we compared *base* with a variant of *base* with outer delayed clausification (*base+outer*) and with a variant with inner delayed clausification (*base+inner*). In the delayed modes, we invoke the Rename rule on formulas that are discovered to occur more than four times in the proof state.

The results show that inner delayed clausification, which performs the laziest form of clausification, gives the worst results on most benchmark sets. Outer delayed clausification performs roughly as well as immediate clausification on problems targeting our logic. On purely first-order problems, it performs a bit worse than immediate clausification. However, outer delayed clausification solves 17 problems not solved by immediate clausification on these problems. This suggests that it opens new possibilities for first-order reasoning that need to be explored further with specialized heuristics and additional rules.

We found a problem with a conjecture of the form $\varphi \rightarrow \varphi$ that only the delayed clausification modes can prove: the TPTP problem `SWV122+1`. The subformula renaming mechanism of immediate clausification obfuscates this problem, whereas delayed clausification allows BoolSimp to convert the negated conjecture to $\bot$ directly, completing the proof in half a second.

To answer question 3, we compared the mode of Zipperposition in which all of the additional rules introduced by our calculus are disabled (*Zip-FO*) with *base*. Our results show that both modes perform roughly the same. Some *base* modes prove up to two more problems within the last seconds of the time allotted due to fluctuations in the evaluation environment beyond our control.

To answer question 4, we evaluated the Boolean selection functions we have implemented: syntactically smallest selectable term (used in *base*), syntactically largest selectable term (sel_{max}), leftmost innermost selectable term (sel_{li}), leftmost outermost selectable term (sel_{lo}), and no Boolean selection ($\mathrm{sel}_{\emptyset}$). We also evaluated two modes in which the rules LocalRw and BoolHoistSimp (BHS) are enabled. None of the selection functions influences the performance greatly. Similarly, we observe no substantial difference regardless of whether the rules LocalRw and BoolHoistSimp are enabled.

8 Related Work and Conclusion

The research presented in this paper extends superposition in two directions: with inprocessing clausification and with first-class Booleans. The first direction has been explored before by Ganzinger and Stuber [12], and others have investigated it in the context of other superposition-related calculi [1, 4, 19, 20].

The other direction has been explored before by Kotelnikov et al., who developed two approaches to cope with first-class Booleans [15, 16]. For the quantified Boolean formula fragment of our logic, Seidl et al. developed a translation into effectively propositional logic [24]. More general approaches to incorporate theories into superposition include superposition for finite domains [13], hierarchic superposition [6], and superposition with (co)datatypes [9].

For SMT solvers [21], supporting first-class Booleans is a widely accepted standard [5]. In contrast, the TPTP TFX format [28], intended to promote first-class Booleans in the rest of the automated reasoning community, is yet to gain traction. Software verification tools could clearly benefit from its popularization, as some of them identify terms and formulas in their logic, e.g., Why3 [11].

In conclusion, we devised a refutationally complete superposition calculus for first-order logic with interpreted Booleans. Its redundancy criterion allows us to flexibly add inprocessing clausification and other simplification rules. We believe our calculus is an excellent choice for the basis of new superposition provers: it offers the full power of regular superposition, while supporting rich input languages such as SMT-LIB and TPTP TFX. Even with unoptimized implementation and basic heuristics, our calculus matches the performance of earlier approaches. In addition, the freedom it offers in term order, literal and Boolean subterm selection opens many possibilities that are yet to be explored. Overall, our calculus is a solid foundation for richer logics in which the Boolean type cannot be efficiently preprocessed, such as higher-order logic [8]. In future work, we plan to tune the heuristics and would find it interesting to combine our calculus with clause splitting techniques, such as AVATAR [30]. This combination amounts to a hybrid approach combining superposition and tableaux.

Acknowledgment Jasmin Blanchette and Martin Desharnais generated the Sledgehammer benchmarks. Simon Cruanes helped us with the implementation. Ahmed Bhayat and Uwe Waldmann suggested textual improvements. The maintainers of StarExec Iowa let us use their service. We thank them all. Nummelin's research has received funding from the Netherlands Organization for Scientific Research (NWO) under the Vidi program (project No. 016.Vidi.189.037, Lean Forward). Bentkamp and Vukmirović's research has received funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation program (grant agreement No. 713999, Matryoshka).

References

[1] Leo Bachmair and Harald Ganzinger. Non-clausal resolution and superposition with selection and redundancy criteria. In Andrei Voronkov, editor, *Logic Programming and Automated Reasoning (LPAR'92)*, volume 624 of *LNCS*, pages 273–284. Springer, 1992.

[2] Leo Bachmair and Harald Ganzinger. Rewrite-based equational theorem proving with selection and simplification. *J. Log. Comput.*, 4(3):217–247, 1994.

[3] Leo Bachmair and Harald Ganzinger. Resolution theorem proving. In John Alan Robinson and Andrei Voronkov, editors, *Handbook of Automated Reasoning*, volume I, pages 19–99. Elsevier and MIT Press, 2001.

[4] Leo Bachmair, Harald Ganzinger, David A. McAllester, and Christopher Lynch. Resolution theorem proving. In *Handbook of Automated Reasoning*, pages 19–99. Elsevier and MIT Press, 2001.

[5] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. The SMT-LIB Standard: Version 2.6. Technical report, Department of Computer Science, The University of Iowa, 2017. Available at `www.SMT-LIB.org`.

[6] Peter Baumgartner and Uwe Waldmann. Hierarchic superposition with weak abstraction. In Maria Paola Bonacina, editor, *CADE-24*, volume 7898 of *LNCS*, pages 39–57. Springer, 2013.

[7] Dan Benanav. Simultaneous paramodulation. In Mark E. Stickel, editor, *CADE-10*, volume 449 of *LNCS*, pages 442–455. Springer, 1990.

[8] Alexander Bentkamp, Jasmin Blanchette, Sophie Tourret, and Petar Vukmirović. Superposition for full higher-order logic. Submitted to CADE-28, https://matryoshka-project.github.io/pubs/hosup_paper.pdf.

[9] Jasmin Christian Blanchette, Nicolas Peltier, and Simon Robillard. Superposition with datatypes and codatatypes. In Didier Galmiche, Stephan Schulz, and Roberto Sebastiani, editors, *IJCAR 2018*, volume 10900 of *LNCS*, pages 370–387. Springer, 2018.

[10] Simon Cruanes. *Extending Superposition with Integer Arithmetic, Structural Induction, and Beyond*. Ph.D. thesis, École polytechnique, 2015.

[11] Jean-Christophe Filliâtre and Andrei Paskevich. Why3—where programs meet provers. In Matthias Felleisen and Philippa Gardner, editors, *European Symposium on Programming (ESOP 2013)*, volume 7792 of *LNCS*, pages 125–128. Springer, 2013.

[12] Harald Ganzinger and Jürgen Stuber. Superposition with equivalence reasoning and delayed clause normal form transformation. *Inf. Comput.*, 199(1-2):3–23, 2005.

[13] Thomas Hillenbrand and Christoph Weidenbach. Superposition for bounded domains. In Maria Paola Bonacina and Mark E. Stickel, editors, *Automated Reasoning and Mathematics*, volume 7788 of *LNCS*, pages 68–100. Springer, 2013.

[14] D. E. Knuth and P. B. Bendix. Simple word problems in universal algebras. In J. Leech, editor, *Computational Problems in Abstract Algebra*, pages 263–297. Pergamon Press, 1970.

[15] Evgenii Kotelnikov, Laura Kovács, Martin Suda, and Andrei Voronkov. A clausal normal form translation for FOOL. In Christoph Benzmüller, Geoff Sutcliffe, and Raúl Rojas, editors, *Global Conference on Artificial Intelligence (GCAI 2016)*, volume 41 of *EPiC*, pages 53–71. EasyChair, 2016.

[16] Evgenii Kotelnikov, Laura Kovács, and Andrei Voronkov. A first class Boolean sort in first-order theorem proving and TPTP. In Manfred Kerber, Jacques Carette, Cezary Kaliszyk, Florian Rabe, and Volker Sorge, editors, *Intelligent Computer Mathematics (CICM 2015)*, volume 9150 of *LNCS*, pages 71–86. Springer, 2015.

[17] Laura Kovács and Andrei Voronkov. First-order theorem proving and vampire. In Natasha Sharygina and Helmut Veith, editors, *Computer Aided Verification (CAV 2013)*, volume 8044 of *LNCS*, pages 1–35. Springer, 2013.

[18] Michel Ludwig and Uwe Waldmann. An extension of the Knuth-Bendix ordering with LPO-like properties. In Nachum Dershowitz and Andrei Voronkov, editors, *Logic Programming and Automated Reasoning (LPAR 2007)*, volume 4790 of *LNCS*, pages 348–362. Springer, 2007.

[19] Zohar Manna and Richard J. Waldinger. A deductive approach to program synthesis. *ACM Trans. Program. Lang. Syst.*, 2(1):90–121, 1980.

[20] Neil V. Murray. Completely non-clausal theorem proving. *Artif. Intell.*, 18(1):67–85, 1982.

[21] Robert Nieuwenhuis, Albert Oliveras, and Cesare Tinelli. Solving SAT and SAT modulo theories: From an abstract Davis–Putnam–Logemann–Loveland procedure to DPLL(T). *J. ACM*, 53(6):937–977, 2006.

[22] Andreas Nonnengart and Christoph Weidenbach. Computing small clause normal forms. In *Handbook of Automated Reasoning*, pages 335–367. Elsevier and MIT Press, 2001.

[23] Visa Nummelin, Alexander Bentkamp, Sophie Tourret, and Petar Vukmirović. Superposition with first-class Booleans and inprocessing clausification (technical report). Technical report, 2021. https://matryoshka-project.github.io/pubs/boolsup_report.pdf.

[24] Martina Seidl, Florian Lonsing, and Armin Biere. qbf2epr: A tool for generating EPR formulas from QBF. In Pascal Fontaine, Renate A. Schmidt, and Stephan Schulz, editors, *Practical Aspects of Automated Reasoning (PAAR-2012)*, volume 21 of *EPiC Series in Computing*, pages 139–148. EasyChair, 2012.

[25] Alexander Steen. *Extensional Paramodulation for Higher-order Logic and Its Effective Implementation Leo-III*. Dissertationen zur künstlichen Intelligenz. Akademische Verlagsgesellschaft AKA GmbH, 2018.

[26] Aaron Stump, Geoff Sutcliffe, and Cesare Tinelli. Starexec: A cross-community infrastructure for logic solving. In *IJCAR 2014*, volume 8562 of *LNCS*, pages 367–373. Springer, 2014.

[27] Geoff Sutcliffe. The TPTP problem library and associated infrastructure—from CNF to TH0, TPTP v6.4.0. *J. Autom. Reason.*, 59(4):483–502, 2017.

[28] Geoff Sutcliffe and Evgenii Kotelnikov. TFX: the TPTP extended typed first-order form. In Boris Konev, Josef Urban, and Philipp Rümmer, editors, *Practical*

Aspects of Automated Reasoning (PAAR-2018), volume 2162 of *CEUR Workshop Proceedings*, pages 72–87. CEUR-WS.org, 2018.

[29] Grigori Tseitin. On the complexity of derivation in propositional calculus. In *Automation of reasoning: Classical Papers on Computational Logic*, volume 2, pages 466–483. Springer, 1983.

[30] Andrei Voronkov. AVATAR: the architecture for first-order theorem provers. In *CAV 2014*, volume 8559 of *LNCS*, pages 696–710. Springer, 2014.

[31] Petar Vukmirović and Visa Nummelin. Boolean reasoning in a higher-order superposition prover. In Pascal Fontaine, Konstantin Korovin, Ilias S. Kotsireas, Philipp Rümmer, and Sophie Tourret, editors, *Practical Aspects of Automated Reasoning (PAAR-2020)*, volume 2752 of *CEUR Workshop Proceedings*, pages 148–166. CEUR-WS.org, 2020.

[32] Petar Vukmirović, Jasmin Blanchette, Simon Cruanes, and Stephan Schulz. Extending a brainiac prover to lambda-free higher-order logic. *Accepted in International Journal on Software Tools for Technology Transfer*.

[33] Uwe Waldmann, Sophie Tourret, Simon Robillard, and Jasmin Blanchette. A comprehensive framework for saturation theorem proving. In Nicolas Peltier and Viorica Sofronie-Stokkermans, editors, *IJCAR 2020*, LNCS. Springer, 2020.